Apache Big Data Lab

Procesamiento distribuido y análisis estadístico de datos bursátiles

Institución: Instituto Superior Santo Domingo.
Carrera: Inteligencia Artificial y Ciencia de Datos.
Materia: Procesamiento de Datos (IAPD).
Evaluación: Segundo Desempeño (TP2).
Año: 2026.
Alumno: Nicolás Druetta.
Matrícula: 100086.
Modalidad: Individual.

---

1. Detalle de la tarea realizada

1.1. Descripción general del proyecto

Apache Big Data Lab es una aplicación desarrollada en Python. Su propósito es implementar un pipeline de procesamiento de datos bursátiles históricos mediante tecnologías de transmisión de eventos, procesamiento distribuido y análisis estadístico.

El proyecto trabaja con información del mercado estadounidense, principalmente operaciones bursátiles (*trades*) y cotizaciones de compra y venta (*quotes*) de acciones dentro del S&P500, obtenidas mediante la API de Alpaca Market Data.

El sistema integra diferentes tecnologías que intervienen en las etapas de adquisición, transmisión, transformación, almacenamiento, análisis estadístico y visualización de datos.

Apache Spark constituye el motor principal de procesamiento. Mediante PySpark y Structured Streaming, los eventos recibidos desde Apache Kafka se procesan en microbatches y se almacenan en formato Parquet para su posterior análisis.

La aplicación utiliza una interfaz desarrollada con Streamlit, que permite configurar la ejecución, observar las etapas del pipeline y consultar los resultados obtenidos.

1.2. Arquitectura del pipeline

El procesamiento se organiza mediante la siguiente secuencia:

Controller → Alpaca → Memoria → Kafka → Streaming → Spark → Parquet → Pandas → PostgreSQL → Streamlit.

Un controlador desarrollado en Python coordina las etapas de ejecución, supervisa su estado y administra las condiciones de inicio, finalización y error.

El controlador no transporta los registros bursátiles entre los componentes: la transferencia de datos desde Alpaca hacia Kafka se realiza directamente.

1.3. Adquisición de datos desde Alpaca

La adquisición se realiza mediante consultas a la API histórica de Alpaca Market Data.

El sistema permite seleccionar los símbolos bursátiles y obtiene automáticamente la última sesión de mercado completa disponible. Para el análisis del índice S&P 500, consulta sus componentes y verifica cuáles están disponibles en Alpaca.

El conjunto de datos es dinámico, ya que los registros obtenidos pueden variar entre ejecuciones según la fecha de consulta y los símbolos seleccionados. El sistema no depende de un dataset estático, sino que permite procesar nuevos datos históricos en cada ejecución.

Los registros se descargan mediante consultas organizadas en grupos de símbolos y ventanas temporales. Cuando una consulta requiere varias páginas de resultados, se recuperan utilizando los mecanismos de paginación de la API.

Los eventos de operaciones y cotizaciones correspondientes a cada ventana se combinan y ordenan cronológicamente antes de entregarse a Kafka. Este procedimiento permite transmitir progresivamente los datos descargados, sin esperar a completar la adquisición de toda la sesión bursátil.

1.4. Publicación de eventos con Apache Kafka

Apache Kafka interviene como plataforma de transmisión de eventos entre la adquisición histórica y el motor de procesamiento.

Los lotes ordenados que entrega Alpaca se publican como mensajes individuales, conservando los atributos de los registros bursátiles y sus marcas temporales originales.

Kafka permite desacoplar la velocidad de descarga de los datos respecto de la velocidad de procesamiento de Spark.

El sistema registra las publicaciones y sus confirmaciones para supervisar el avance de la transmisión.

1.5. Procesamiento distribuido con Apache Spark

Apache Spark es el componente central del proyecto como motor de procesamiento distribuido.

La aplicación utiliza Spark Structured Streaming para consumir los mensajes publicados en Apache Kafka y procesarlos mediante microbatches, es decir, pequeños lotes de eventos que se procesan progresivamente.

El procesamiento se organiza según las etapas ETL (Extract, Transform, Load) y comprende las siguientes operaciones:

- Extracción (Extract): lectura de los eventos publicados en Apache Kafka.
- Transformación (Transform): interpretación y estructuración de los registros mediante DataFrames de PySpark, validación de datos, tratamiento de registros inválidos y eliminación de registros duplicados dentro de cada microbatch.
- Carga (Load): escritura de los registros procesados en archivos Apache Parquet.

La arquitectura dispone de un **Spark Master** y **dos Spark Workers**. El Master administra los recursos del clúster, mientras que los Workers ejecutan las tareas de procesamiento distribuidas por Spark.

Esto permite dividir el trabajo en particiones y procesar diferentes conjuntos de registros en paralelo, según los recursos disponibles.

Spark también utiliza mecanismos de checkpoint para conservar información sobre el avance del procesamiento y facilitar su recuperación ante interrupciones.

Spark realiza un análisis exploratorio inicial mediante DataFrames de PySpark, calculando la cantidad de eventos procesados, operaciones bursátiles (trades), cotizaciones (quotes) y símbolos bursátiles diferentes. Estos indicadores permiten supervisar la composición y el volumen de los datos procesados. Posteriormente, Pandas desarrolla el análisis estadístico de los datos almacenados.

1.6. Almacenamiento en Apache Parquet

Los eventos procesados por Spark se almacenan en archivos Parquet.

Este formato permite conservar información estructurada y facilita la lectura posterior mediante herramientas compatibles con datos columnares.

En el proyecto, los archivos Parquet se usan como intermediarios entre el procesamiento distribuido realizado por Spark y el análisis estadístico ejecutado mediante Pandas.

1.7. Análisis estadístico con Pandas

Pandas consume los datos disponibles en Parquet y calcula indicadores asociados con la actividad bursátil.

Entre las métricas obtenidas se encuentran:

- Cantidad de operaciones por símbolo.
- Cantidad de retornos calculados.
- Volatilidad de los precios.
- Cantidad de observaciones del spread.
- Spread promedio entre compra y venta.
- Volumen total negociado.

También se calculan correlaciones estadísticas relacionadas con las dos preguntas de investigación planteadas en el proyecto.

1.8. Persistencia de resultados en PostgreSQL

PostgreSQL almacena los resultados estadísticos del procesamiento.

La persistencia se organiza principalmente en dos tablas:

- market_metrics: almacena las métricas calculadas para cada símbolo bursátil.
- market_answers: almacena los resultados agregados correspondientes a las preguntas de investigación.

El sistema utiliza operaciones UPSERT para insertar resultados nuevos y actualizar registros existentes, evitando duplicar filas que correspondan a las mismas claves.

Cada ejecución se identifica mediante un run_id, que permite relacionar los resultados almacenados con el procesamiento que los produjo.

1.9. Visualización y supervisión mediante Streamlit

La aplicación web permite configurar el análisis, iniciar y detener la ejecución y observar los registros generados por las etapas.

La interfaz presenta seis consolas independientes correspondientes al controlador, Alpaca, Kafka, Spark, Pandas y PostgreSQL.

Adicionalmente, muestra gráficos de actividad bursátil, indicadores estadísticos y las respuestas a las preguntas de investigación.

El sistema incorpora mecanismos de supervisión y control de errores para evitar que una ejecución continúe cuando falla un componente crítico.

---

2. Herramientas y tecnologías empleadas

Tecnología	**Función dentro del proyecto**	**Motivo de utilización**
Python	Desarrollo de los módulos	Lenguaje de integración y procesamiento de datos.
Alpaca Market Data	Adquisición de información bursátil	Acceso programático a datos históricos.
Apache Kafka	Transmisión de eventos.	Desacoplamiento entre adquisición y procesamiento.
Apache Spark / PySpark	Procesamiento de datos	Motor distribuido y escalable.
Spark Structured Streaming	Procesamiento continuo	Consumo y transformación mediante microbatches.
Apache Parquet	Almacenamiento de datos procesados	Formato columnar.
Pandas	Análisis estadístico	Cálculo de indicadores y correlaciones.
PostgreSQL	Persistencia relacional	Almacenamiento y actualización de resultados.
Streamlit	Interfaz web	Supervisión y presentación de información.
Docker Compose	Infraestructura	Coordinación de servicios y contenedores.

Justificación de la arquitectura y decisiones de diseño

Durante el desarrollo del proyecto, evalué distintas formas de organizar el pipeline. Consideré procesar los datos directamente desde Alpaca, pero decidí incorporar Apache Kafka para separar la descarga del procesamiento y permitir que Spark consuma los eventos a su propio ritmo.

También opté por dividir el almacenamiento y el análisis en distintas etapas: Parquet para guardar los datos procesados, Pandas para calcular las estadísticas, en lugar de realizar todo el análisis con PySpark, y PostgreSQL para almacenar los resultados finales. Esta decisión me permitió separar el procesamiento distribuido del análisis estadístico y simplificar los cálculos.

El resultado es una arquitectura modular en la que cada herramienta tiene una responsabilidad concreta, lo que facilita identificar errores y trabajar con grandes cantidades de datos.

---

3. Motivos

Elegí trabajar con el mercado bursátil estadounidense porque me interesa el mundo de las inversiones y quería aprovechar ese interés para desarrollar un proyecto con datos reales. Además, es un mercado que genera una gran cantidad de operaciones y cotizaciones, lo que permite trabajar con volúmenes importantes de información.

Los datos bursátiles contienen precios, cantidades, símbolos y marcas temporales que permiten realizar validaciones, transformaciones y cálculos estadísticos. También me interesaba analizar si existe alguna relación entre la cantidad de operaciones, la volatilidad de los precios y el spread de las acciones.

Uno de los principales desafíos fue diseñar una arquitectura que pudiera descargar, transmitir y procesar todos estos datos sin depender de una única etapa. Por eso incorporé Apache Kafka entre Alpaca y Spark, para que la descarga y el procesamiento funcionaran de manera independiente.

También decidí separar el procesamiento distribuido del análisis estadístico. Spark se encarga de limpiar, transformar y almacenar los registros en Parquet, mientras que Pandas calcula los indicadores y correlaciones. Finalmente, PostgreSQL guarda los resultados y Streamlit permite visualizar y supervisar todo el proceso.

Durante el desarrollo tuve que resolver dificultades relacionadas con la descarga de grandes cantidades de registros, la transmisión progresiva de eventos, la coordinación entre componentes y el control de errores. Estas dificultades me llevaron a realizar varios ajustes hasta llegar a la arquitectura actual.

---

4. Objetivos

4.1. Objetivo general

Diseñar e implementar un pipeline de ingeniería de datos que permita adquirir, transmitir, procesar, analizar y visualizar información bursátil histórica mediante Apache Spark y herramientas complementarias.

4.2. Objetivos específicos

- Obtener datos históricos de operaciones y cotizaciones desde una API externa.
- Transmitir los eventos mediante Apache Kafka.
- Procesar los datos utilizando DataFrames de PySpark y Structured Streaming.

- Aplicar procedimientos de validación, transformación y deduplicación.
- Almacenar los registros procesados en formato Parquet.
- Calcular indicadores estadísticos utilizando Pandas.
- Persistir los resultados en PostgreSQL.
- Presentar los resultados y supervisar el pipeline desde Streamlit.
- Evaluar la relación entre actividad bursátil, volatilidad y spread.

4.3. Preguntas de investigación

Pregunta 1: ¿Las acciones con más operaciones presentan mayor volatilidad?

Analizo la relación estadística entre la cantidad de operaciones registradas por símbolo y la volatilidad calculada a partir de sus precios.

Pregunta 2: ¿Las acciones con más operaciones presentan menor spread entre compra y venta?

Analizo la relación estadística entre la cantidad de operaciones registradas por símbolo y el spread promedio de sus cotizaciones.

---

5. Hallazgos

Durante el desarrollo del proyecto realicé diferentes ejecuciones para comprobar el funcionamiento de la arquitectura y el procesamiento distribuido de los datos. Como una sesión bursátil completa del S&P 500 puede requerir varias horas de procesamiento, las pruebas se realizaron con ejecuciones parciales.

Uno de los principales resultados fue lograr que Alpaca, Kafka y Spark trabajaran de manera coordinada, permitiendo descargar, transmitir y procesar grandes cantidades de eventos sin esperar a que finalizara la descarga completa.

Apache Kafka permitió separar la adquisición de datos del procesamiento. Los eventos se publicaron en diferentes particiones y Spark los consumió mediante Structured Streaming, trabajando con microbatches.

En Spark pude observar la distribución de tareas entre los dos Workers. El procesamiento incluyó la lectura de eventos, validación de registros, eliminación de duplicados y escritura de archivos Parquet. También fue posible supervisar la cantidad de registros procesados y las tareas ejecutadas.

Otro resultado importante fue la integración de Pandas y PostgreSQL. A medida que Spark completaba los lotes, Pandas podía calcular los indicadores estadísticos y PostgreSQL almacenar los resultados, sin necesidad de esperar a que terminara toda la sesión bursátil.

La interfaz desarrollada con Streamlit me permitió supervisar las seis etapas del pipeline mediante consolas independientes, consultar los resultados y detectar errores durante las ejecuciones.

En general, las pruebas parciales permitieron comprobar el funcionamiento de la arquitectura distribuida, la comunicación entre los componentes y el procesamiento progresivo de los datos. La principal limitación fue el tiempo necesario para procesar una sesión bursátil completa con una cantidad elevada de símbolos.

6. Conclusiones

El desarrollo de Apache Big Data Lab me permitió aplicar los conocimientos adquiridos durante la materia e integrar diferentes tecnologías en un pipeline de procesamiento distribuido.

Considero que cumplí con el objetivo principal del proyecto, ya que logré implementar una arquitectura capaz de adquirir datos bursátiles desde Alpaca, transmitirlos mediante Kafka, procesarlos con Apache Spark y almacenar los resultados para su posterior análisis con Pandas y PostgreSQL.

Uno de los aspectos más importantes fue trabajar con Apache Spark y Structured Streaming, comprender cómo funcionan los microbatches y observar la distribución de tareas entre los Workers. También pude comprobar la importancia de separar las responsabilidades de cada componente para facilitar el procesamiento y la supervisión del sistema.

Durante el desarrollo encontré dificultades relacionadas con la descarga de grandes volúmenes de datos, la comunicación entre los servicios, la sincronización de las etapas y el manejo de errores. Resolver estos problemas fue una parte importante del aprendizaje y me llevó a realizar varios ajustes en la arquitectura.

La principal limitación fue el tiempo necesario para procesar una sesión bursátil completa del S&P 500, por lo que las ejecuciones realizadas fueron parciales. Sin embargo, permitieron comprobar el funcionamiento del pipeline y obtener resultados a partir de datos reales.

Como mejoras futuras, me gustaría optimizar los tiempos de procesamiento, evaluar el comportamiento del sistema con mayores volúmenes de datos y ampliar las herramientas de análisis y supervisión.

Autor: Nicolás Druetta.